AIFX Studio Phase 2

Face Enhancement API Four-Day Delivery Plan

PDF-aligned implementation plan for ComfyUI API integration, LoRA mapping, spatial merging, and validation.

Target application: AIFX Studio Integration

Phase 2 deadline: July 13, 2026

Prepared by: Zooey Chen

Date: 2026-07-07

Workflow template: ComfyUI API prompt template: zooey.json

Immediate test case: Submit one cropped face image to ComfyUI and return one enhanced cropped face image

Executive Summary

This plan matches the Phase 2 assignment. The implementation will extend the Phase 1 face detection output, route cropped face regions through a ComfyUI API workflow with character-specific LoRAs, and merge enhanced faces back into the original high-resolution image. The first practical test will use one crop image to validate the ComfyUI API loop before expanding to multi-face processing and final spatial blending.

1. Assignment Alignment

PDF Requirement	Status	Plan Coverage
ComfyUI API wrapper	Covered	Use HTTP/WebSocket API, /prompt submission, history polling, and output retrieval.
Dynamic JSON construction	Covered	Use zooey.json as a fixed template and inject image, LoRA, prompt, and output prefix at runtime.
Character-to-LoRA mapping	Covered	Create lora_config.json or a table mapping character_id/name to exact LoRA filenames.
Multi-character handling	Covered	Loop through each detected face and call ComfyUI sequentially first, with parallel execution as an optimization.
Spatial merging and blending	Covered	Use Phase 1 bbox metadata and blend the enhanced crop using alpha feathering, mask blur, or Poisson blending.
Validation UI	Covered	Extend the Phase 1 UI with input, character assignment, logs, and visual comparison.
Standalone API endpoint	Covered	Provide one protected POST endpoint for the automated Phase 1 + Phase 2 chain.

2. Project Objective

The Phase 2 objective is to turn the Phase 1 face detection prototype into an API-driven face enhancement pipeline. The service will receive an original image or detected crop data, identify the correct character LoRA, submit each cropped face region to ComfyUI, retrieve the enhanced face output, and merge the result back into the original image without visible bounding-box edges.

- Immediate validation: one crop image in, one enhanced crop image out.
- Final Phase 2 behavior: original image plus detected faces in, final blended high-resolution output image out.
- Workflow JSON must not be edited manually during testing; runtime values are injected by backend code.

3. Workflow Automation Strategy

The exported ComfyUI API JSON will be stored as a workflow template. For every API request, the backend creates a deep copy of the template and injects request-specific values into known node inputs before submitting the payload to ComfyUI.

Node ID	Class	Field	Runtime Value
958	LoadImage	inputs.image	Uploaded cropped face image filename
1056	LoraLoaderModelOnly	inputs.lora_name	LoRA filename resolved from character_id
1057	LoraLoaderModelOnly	inputs.lora_name	Same character LoRA, unless workflow-specific mapping differs
1071	Text Multiline	inputs.text	Optional prompt or enhancement instruction
866	SaveImage	inputs.filename_prefix	Unique request_id or job_id output prefix

4. API Design

4.1 Minimum Crop Test Endpoint

```
POST /api/v1/face-enhance
Authorization: Bearer <api_key>
Content-Type: multipart/form-data

image: <cropped_face_image.jpg>
character_id: <character_or_lora_key>
prompt: <optional enhancement instruction>
```

4.2 Full Phase 2 Endpoint

```
POST /api/v1/face-swap
Authorization: Bearer <api_key>
Content-Type: multipart/form-data
```

```
image: <original_high_resolution_image.jpg>
faces: [
  {
    "face_id": "face_001",
    "bbox": {"xmin": 120, "ymin": 80, "width": 256, "height": 256},
    "character_id": "cousin_sean"
  }
]
```

4.3 Expected Response

```
{
  "job_id": "req_20260707_001",
  "status": "completed",
  "enhanced_crop_url": "https://.../outputs/req_20260707_001_crop.png",
  "final_image_url": "https://.../outputs/req_20260707_001_final.png",
  "metadata": {
    "workflow_template": "zooey.json",
    "comfyui_prompt_id": "...",
    "blend_method": "alpha_feathering"
  }
}
```

5. Four-Day Delivery Plan

Day	Focus	Main Tasks	Deliverables
Day 1	Workflow extraction and one-crop ComfyUI test	Confirm zooey.json is the API-format workflow template. Create node mapping. Upload one crop image to ComfyUI. Inject node 958, 1056, 1057, 1071, and 866. Submit /prompt and retrieve one enhanced crop output.	Single crop test passes without manually editing JSON. Node mapping and test evidence are documented.
Day 2	API wrapper, LoRA mapping, and account protection	Build the Python ComfyUI client and protected POST endpoint. Add lora_config.json mapping character_id/name to exact LoRA filenames. Implement API key/account authentication and basic error responses.	A caller can submit one crop image through API and receive an enhanced crop. Invalid API keys are rejected.
Day 3	Multi-face loop, GCP hosting, and production-like test	Deploy API and ComfyUI on GCP or a selected host platform. Add multi-character loop over detected faces. Confirm model, VAE, LoRA, and custom node availability. Test public API access with API key protection.	Hosted endpoint works. Multiple detected faces can be processed sequentially. ComfyUI timeout and LoRA loading errors are handled.
Day 4	Spatial merging, validation UI, and handoff	Use Phase 1 bbox metadata to place enhanced faces back into the original image. Implement alpha feathering or Poisson blending. Extend the validation UI with upload, character assignment, execution logs, and comparison matrix.	Final image shows no visible square crop boundary. Handoff includes API docs, deployment notes, test screenshots, and known limitations.

6. Day 1 Immediate Test Flow

1. Prepare one cropped face image, for example test_crop_001.jpg.
2. Upload the crop to ComfyUI and capture the server-side image filename.

3. Load zooey.json and deep copy it into a request payload.
4. Inject the uploaded filename into node 958 inputs.image.
5. Resolve character_id to the correct LoRA filename and inject it into nodes 1056 and 1057.
6. Inject optional prompt text into node 1071.
7. Set node 866 filename_prefix to a unique job_id.
8. Submit the payload to ComfyUI /prompt and poll /history until the output is available.
9. Return the enhanced crop image and log request_id, prompt_id, LoRA, output path, and runtime.

7. Spatial Merging and Blending Plan

For the final Phase 2 output, the enhanced face cannot be pasted directly onto the original image. The backend must retrieve Phase 1 bounding-box metadata, resize the enhanced crop to the exact target region, and blend it into the source image using a soft mask.

- Required metadata: xmin, ymin, width, height, face_id, and character_id.
- Initial method: PIL alpha feathering with Gaussian-blurred mask edges.
- Higher-quality option: OpenCV seamlessClone / Poisson blending when lighting or skin tone mismatch is visible.
- Acceptance standard: no visible square crop boundary and no harsh skin-tone discontinuity around the face region.

8. Validation UI Expansion

UI Area	Requirement
Input Panel	Image upload and manual character identity assignment for each detected face.
Execution Logs	Real-time states such as Queueing, Processing, ComfyUI completed, Blending, and Failed.
Comparison Matrix	Side-by-side view of original image, isolated ComfyUI face output, and final merged image.
Debug Metadata	Show face_id, bbox, selected LoRA, ComfyUI prompt_id, and output file path for validation.

9. Risks and Mitigation

Risk	Impact	Mitigation
MediaPipe/face detection misses faces	Production must not skip faces	Keep InsightFace or a hybrid detector as fallback and process all Phase 1 detected regions.
Workflow node IDs change	Runtime injection fails	Store node mapping in config and validate required nodes at service startup.
LoRA file missing or fails to load	ComfyUI job fails	Validate lora_config.json against server files and return structured errors.
ComfyUI timeout or worker crash	API request hangs or fails	Set timeout, job status polling, retry policy, and clear failure response.

Risk	Impact	Mitigation
Poor crop boundary blending	Final image looks visibly edited	Use feathered masks first, then Poisson blending for difficult lighting cases.
Public endpoint abuse	Security and cost risk	Expose only the API service, require API key/account, and keep ComfyUI UI private.

10. Acceptance Criteria

- A single protected POST endpoint can automate the Phase 1 + Phase 2 chain.
- The crop-only test returns an enhanced cropped face image through the API.
- The backend dynamically injects crop image and LoRA filename into the ComfyUI JSON payload.
- Character-to-LoRA mapping is stored in lora_config.json or equivalent configuration.
- Multiple detected faces can be processed one by one, with the correct LoRA for each character.
- Enhanced faces are merged back into original coordinates without visible square boundaries.
- Validation UI shows original image, isolated ComfyUI output, final merged image, and execution logs.
- Timeouts, missing LoRA files, and ComfyUI failures do not crash the server.